experiment to see how effectively the system handles both straightforward and ambiguous questions.

6.3 Using memory components in LangChain to remember conversation history

So far we have kept the history of messages in memory by using the `getMutableAIState` to keep the AI state in sync. But this approach lacks persistence and does not provide mechanisms for querying different conversational sessions, leaving it up to the developer to implement a suitable solution.

To be fair, the Vercel AI SDK provides an alternative method for storing and retrieving conversation history through callbacks. Specifically, it includes the `onFinish` callback of the `generateText` or `streamText` functions. This callback is triggered after the model's response and all tool executions are complete, providing access to the final text, tool calls, tool results, and usage information. This makes it an ideal point to store chat history in a database. Despite this helpful feature, developers still need to implement their own integration with a storage solution to manage conversation history effectively.

Using LangChain, developers have access to a set of tools to manage conversation history with proper abstractions and tools to verify the semantic meaning of them. Figure 6.6 shows how to implement conversational memory components using LangChain.

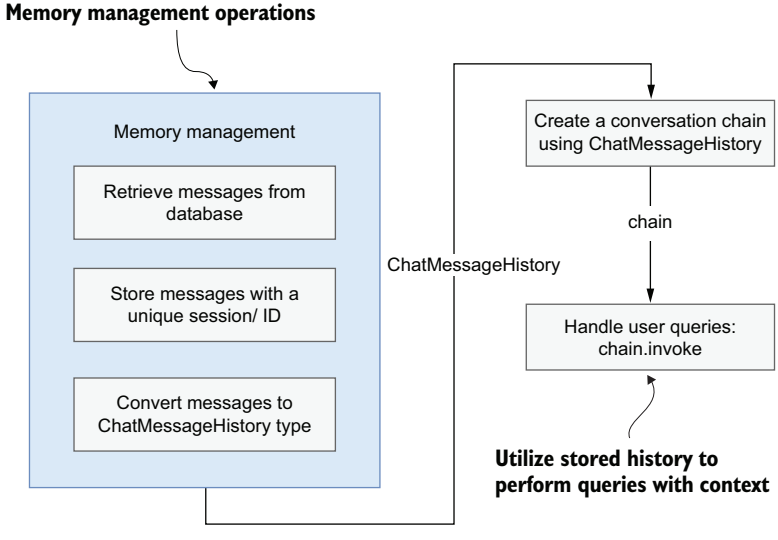


Figure 6.6 Managing user messages to improve conversation accuracy. Messages are retrieved from a database, stored with a unique session ID, and converted into a `ChatMessageHistory` format. This history is used to create a `ConversationChain` that handles user queries, using stored context to enhance response accuracy. The history is integrated into other prompts to refine the model's performance.